Credit Risk Modelling

Akshay Kulkarni, Nima Marvi, Steven Ly

1.Executive Summary

Project Intro –

- We are a bank that handles consumer loans.
- According to the St. Louis Fed, the average delinquency rate for Credit Card loans given by commercial banks from Q2 2017 – Q2 2018 was 2.50%.
- This project aims to develop a machine learning model that predicts whether a credit card customer will default within 12 months of their most recent statement. The model leverages historical behavior to help identify risky customer profiles.

Project Goals –

- Accurately classify customers as likely defaulters or non-defaulters
- Enhance decision-making for credit risk and collections teams
- Build a credit risk model and design a strategy to assign credit

Why this project?

- Rising credit card usage increases exposure to default risk
- Early prediction of default enables smarter financial planning
- Timely intervention can reduce delinquency and improve recovery

How does it help the company?

- Minimize financial losses by targeting high-risk customers early by optimizing resource allocation in collections and customer service
- Strengthen risk management and compliance frameworks will improve profitability by retaining reliable customers and reducing write-offs

Result of your Strategies

	Train			**Test1**			**Test2**		
	# Total	Default Rate	Revenue	# Total	Default Rate	Revenue	# Total	Default Rate	Revenue
Conservative Strategy (0.5)	39340	0.00061	502	8335	0.02	100.7	8335	0.02	106
Aggressive Strategy (0.9)	48228	0.06	579	10415	0.08	118.4	10345	0.09	122

2. Data

Why did we decide to use this data?
We selected historical data from March 2018 to train our model on a stable economic period.

- During this time, delinquency rates were relatively low, making the data more balanced and reliable.
- We aimed to minimize the impact of external shocks (e.g., post-2018 financial uncertainty or global disruptions)
- This helped us capture true customer behavior, free from the noise or biases that might arise during economic turmoil

Data Exploration:

- The dataset includes 458K+ unique customers, each with up to 13 months of behavioral history
- Rich variables spanning spend, payments, balances, risk flags, and more

Target variable:

- Binary outcome: 1 if the customer defaulted within 12 months after March 2018, else 0
- Mirrors real-world risk prediction used in credit underwriting

Category	# Observations	Default Rate
All Applications	1,106,290	24.82%
Applications with 13 months of historical data	66,667	23.54%
Applications with 12 months of historical data	102,000	23.43%
Applications with 11 months of historical data	116,772	23.83%
Applications with 10 months of historical data	118,608	23.78%
Applications with 9 months of historical data	114,508	23.99%
Applications with 8 months of historical data	104,463	24.77%
Applications with 7 months of historical data	94,042	24.99%
Applications with 6 months of historical data	81,554	25.60%
Applications with 5 months of historical data	69,277	25.23%
Applications with 4 months of historical data	57,532	26.20%
Applications with 3 months of historical data	48,703	26.52%
Applications with 2 months of historical data	40,746	26.88%
Applications with 1 months of historical data	91,418	27.63%

3.Features

Notable Variables (Raw Data):

- Delinquency - Tracks missed payments and late payment indicators (e.g., DUE_AMT, PAST_DUE_DAYS)
- Spend - Captures spending activity (e.g., SPEND_AMT, TXN_COUNT)
- Payment - Includes payments made, due amounts, and utilization (e.g., PAYMENT_AMT, MIN_DUE_AMT)
- Balance - Reflects current balance, credit usage, and limits (e.g., BALANCE, CREDIT_LIMIT)
- Risk - Flags related to risk (e.g., MISSED_PAYMENTS, UTILIZATION_RATIO)

Rationale for Using These Variables

- These categories collectively represent a customer's financial behavior, creditworthiness, and risk exposure.
- Provided a comprehensive and interpretable foundation for downstream feature engineering
- Ensured we could track behavioral patterns over time — key for building robust risk models

Category	# of Features
D_* = Delinquency variables	96
S_* = Spend variables	22
P_* = Payment variables	3
B_* = Balance variables	40
R_* = Risk variables	28

4. Feature Engineering

Numerical Aggregation:

- For all numerical features, created aggregation values for each customer based on mean, sum, min, max, and std.
- Started with 177 numerical features
- After aggregation, 885 numerical features

Categorical Aggregation:

- Created new columns based on Response Rate & Ever Response (over course of 3,6,9,12 months respectively)
- Response Rate = Average amount of times categorical variable = 1
- Ever Response Rate = Amount of times categorical variable = 1
- 11 initial categorical features

5 Features with highest SHAP

Feature	Min	1 Percentile	5 Percentile	Median	95 Percentile	99 Percentile	Max	Mean	% Missing
P_2_mean_3m	-0.374	-0.041	0.166	0.683	0.972	1.00	1.01	0.639	0.612
P_2_min_3m	0.421	-0.102	0.110	0.660	0.959	1.001	1.01	0.614	0.612
D_42_min_12m	-0.000281659	0.00095	0.0037	0.102	0.528	0.961	4.185	0.165	76.269
R_1_mean_3m	6.44 E -7	0.0014	0.0024	0.00567	0.588	1.170	2.175	0.098	0
B_2_min_3m	5.48 E -8	0.00069	0.0039	0.811	1.00	1.01	1.01	0.530	0.00623

5.Data Processing / One-Hot Encoding

Categorical Variables

- Mix of Balance and Delinquency Variables

  - Given on Kaggle data summary

['B_30', 'B_38', 'D_114', 'D_116', 'D_117', 'D_120', 'D_126', 'D_63', 'D_64', 'D_66', 'D_68']

Treatment:

- We converted these categorical variables to binary values (0 and 1)
- Done using built-in "get_dummies" pandas function
- 34 newly created columns

```python
# ------ Step 2: One-Hot Encoding ------ #
# Specify categorical columns for one-hot encoding
categorical_cols = ['B_30', 'B_38', 'D_114', 'D_116', 'D_117', 'D_120', 'D_126', 'D_63', 'D_64', 'D_66', 'D_68']
sample_data_encoded = pd.get_dummies(sample_data, columns=categorical_cols, drop_first=True, dtype=int)

# Track newly created columns
newly_created_columns = [
    col for col in sample_data_encoded.columns.tolist()
    if col not in sample_data.columns
```

Before One-Hot Encoding

D_126	D_66	D_68
0	1	3

After One-Hot Encoding

D_68_1.0	D_68_2.0	D_68_3.0...
0	1	1

6. Feature Selection

- We narrowed down our features to only features with importance of > 0.5% Predictive Power

- Ran 2 XGB models on the train sample: first with default parameters, and a second with the following parameters:

300 trees
0.5 learning rate
max depth of trees of 4
50% of obs to build each tree
50% of features to build each tree
weight of 5 to default obs.

- Result of 18 important features

- ['P_2_mean_9m', 'P_2_mean_3m', 'P_2_max_3m', 'B_1_mean_3m', 'B_11_min_3m', 'P_2_min_3m', 'B_2_mean_3m', 'B_9_mean_3m', 'S_23_max_3m', 'B_3_min_9m', 'B_1_min_3m', 'B_7_max', 'R_1_mean_6m', 'B_23_std', 'D_44_mean_3m', 'D_48_mean_9m', 'B_3_std_6m', 'B_9_max_3m']

Excel files with results of feature importance for two models (steps 8 and 9)

Feature Importance 1

Microsoft Excel Worksheet

Feature Importance 2

Microsoft Excel Worksheet

Category	# of Features	# selected
D_* = Delinquency variables	2037	2
S_* = Spend variables	441	1
P_* = Payment variables	63	4
B_* = Balance variables	862	10
R_* = Risk variables	589	1

7.XGBoost – Grid Search

What is Grid Search?

- Grid Search uses XGBoost to find the best hyperparameters used to create the most optimal ML model.
- In this scenario, we are using AUC (area under the curve) to evaluate model performance.

Grid Search Parameters:

- Chosen as they are commonly used.

  - Number of trees: 50, 100, and 300
  - Learning Rate: 0.01, 0.1
  - Percentage of observations used in each tree: 50%, 80%
  - Percentage of features used in each tree: 50%, 100%
  - Weight of default observations: 1, 5, 10

How many models have you trained?

- 72 total models trained using XGB grid search

```python
# Loop over the combinations manually (without using a param_grid dictionary)
for n_estimators in [50, 100, 300]:  # Number of trees
    for learning_rate in [0.01, 0.1]:  # Learning rates
        for subsample in [0.5, 0.8]:  # Percentage of observations used in each tree
            for colsample_bytree in [0.5, 1.0]:  # Percentage of features used in each tree
                for scale_pos_weight in [1, 5, 10]:  # Weight of default observations

                    # Set up the model with the current hyperparameters
                    model = xgb.XGBClassifier(
                        n_estimators=n_estimators,
                        learning_rate=learning_rate,
                        subsample=subsample,
                        colsample_bytree=colsample_bytree,
                        scale_pos_weight=scale_pos_weight,
                        missing=np.nan,
                        use_label_encoder=False
                    )

                    # Train the model
                    model.fit(train_df, train_y)

                    # Get the AUC score on training and both testing sets
                    y_train_pred = model.predict_proba(train_df)[:, 1]  # Predicted probabilities for AUC
                    y_test1_pred = model.predict_proba(test1_df)[:, 1]  # Predicted probabilities for AUC (Test 1)
                    y_test2_pred = model.predict_proba(test2_df)[:, 1]  # Predicted probabilities for AUC (Test 2)

                    auc_train = roc_auc_score(train_y, y_train_pred)
                    auc_test1 = roc_auc_score(test1_y, y_test1_pred)
                    auc_test2 = roc_auc_score(test2_y, y_test2_pred)

                    # Save the results for the current iteration
                    results.append({
                        'Trees': n_estimators,
                        'LR': learning_rate,
                        'Subsample': f'{int(subsample*100)}%',
                        '% Features': f'{int(colsample_bytree*100)}%',
                        'Weight of Default': scale_pos_weight,
                        'AUC Train': auc_train,
                        'AUC Test 1': auc_test1,
                        'AUC Test 2': auc_test2
```

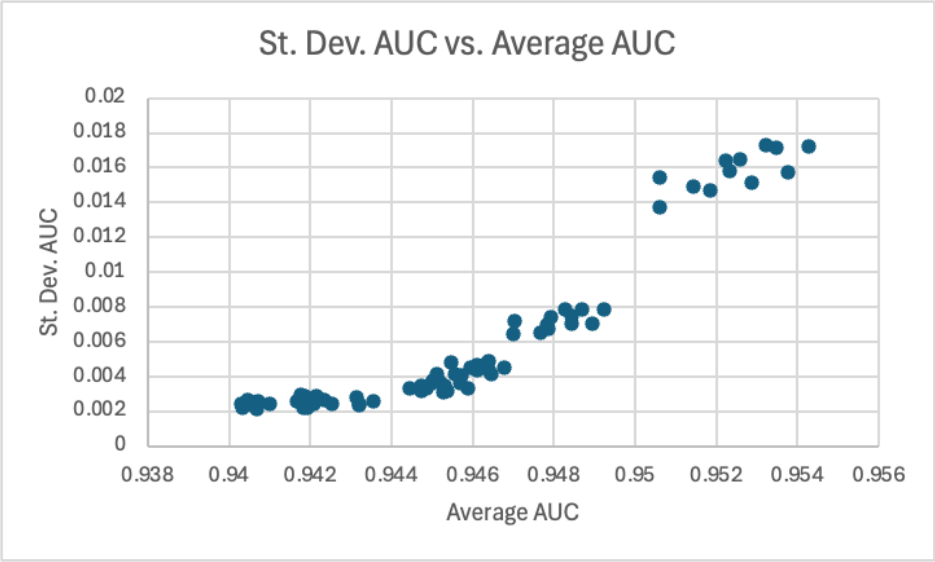
8.XGBoost – Grid Search

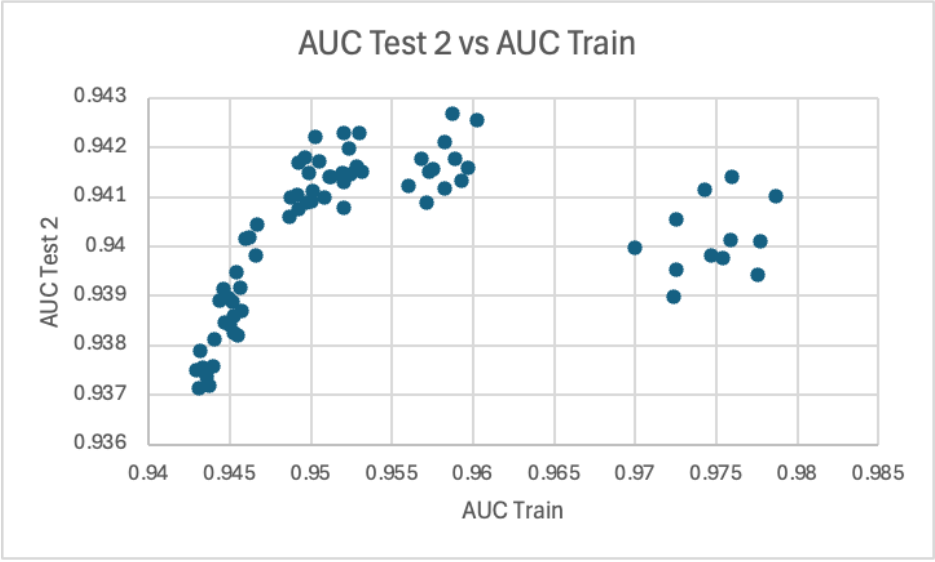
- Based on Scatterplot 1, we would likely choose a model that is closer to the center of the graph (highest avg AUC & lowest St. Dev. AUC)

- Low St.Dev AUC Across Train + Test Samples = Low Bias/Variance
- High average AUC Across Train + Test Samples = High model classification performance

- Based on Scatterplot 2, we would likely choose a model closer to the bottom left of the scatterplot (lowest difference between Train AUC and Test 2 AUC)

- Prefer a low difference between Test/Train AUC to indicate low model bias





9.XGBoost – Final Model

Optimal Model Characteristics:

1. Low Variance (Between Test 1 & Test 2 Data)
2. Low Bias (Between Train & Test Data)
3. High AUC (Across all 3 Sets)

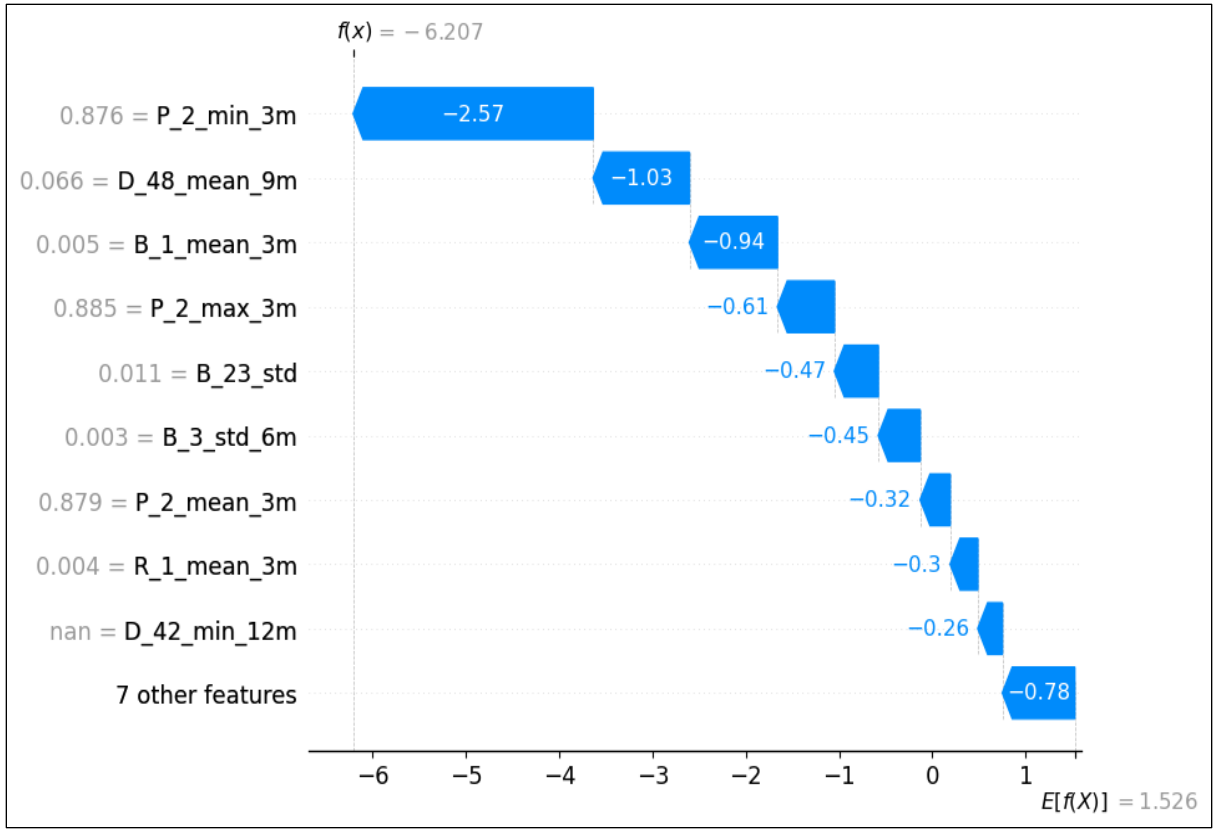
Model Parameters	
Trees	300
Learning Rate	0.01
Subsample	50%
% Features	100%
Weight of Default	1

AUCs of Optimal XGB Model	
AUC Train	0.950
AUC Test 1	0.945
AUC Test 2	0.942
AVG AUC Test	0.944
AUC Test Variance	0.000017

10,11.XGBoost – SHAP Analysis

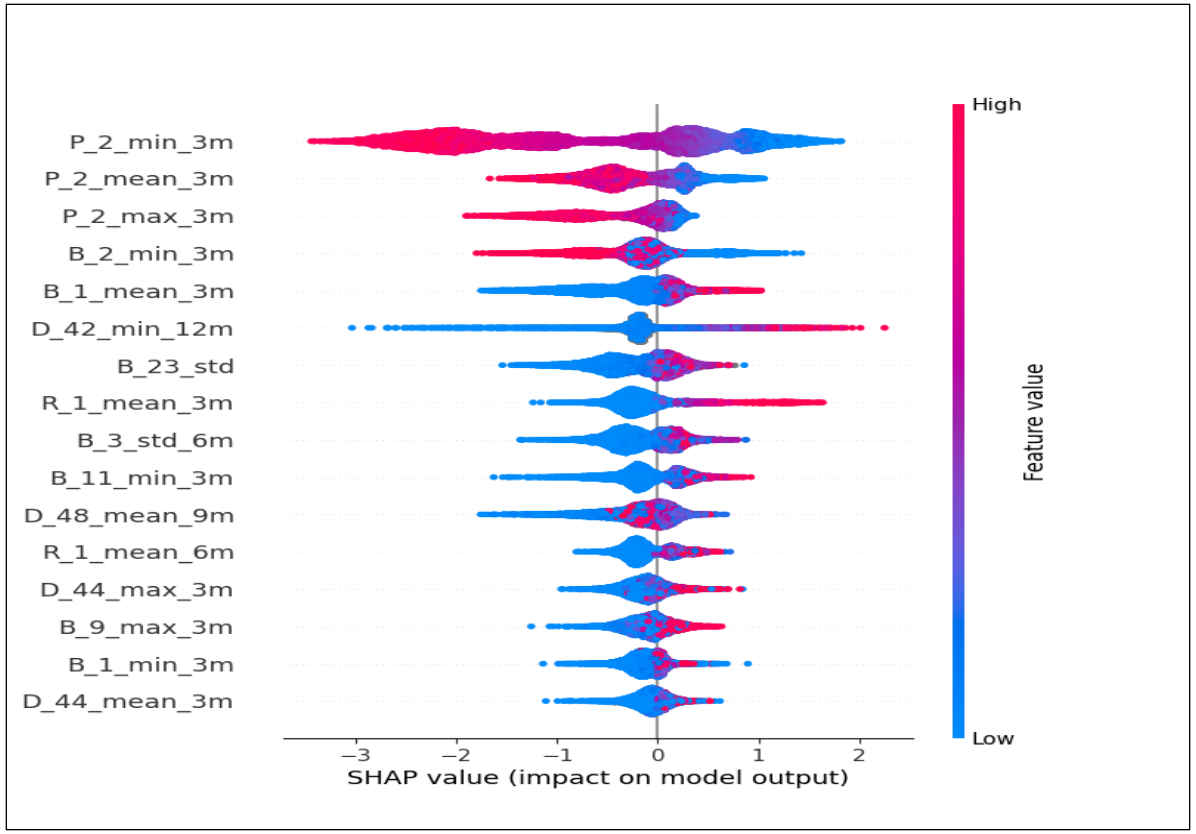
Waterfall Interpretation:

- Larger Boxes = higher features importance
- Smaller Boxes = lower features importance



Beeswarm Graph Interpretation:

- Redder Values = higher feature importance
- Bluer Values = Lower feature importance



12.Neural Network – Data Processing

Neural Network Data Processing:

Missing value treatment, we replaced missing values with 0.

```
# Step 1: Handle missing values and replace them with 0 (SimpleImputer)

imputer = SimpleImputer(strategy='constant', fill_value=0)
X_train_imputed = imputer.fit_transform(train_df)
X_test1_imputed = imputer.transform(test1_df)
X_test2_imputed = imputer.transform(test2_df)
```

13.Neural Network – Grid Search

NN Grid Search Parameters:

- Number of hidden layers: 2, 4
- # nodes in each hidden layer: 4, 6
- Activation function for hidden layers: ReLu, Tanh
- Dropout regularization for hidden layers: 50%, 100% (no dropout)
- Batch size: 100, 10000

32 NN Models Trained

Chosen to control for overfitting/computational processing:
- Number of hidden layers
- # Nodes in each hidden layer
- Dropout regularization
- Batch Size

Relu

Default activator
Fast training, and deep network

Tanh

Alternative activator
Shallow network
Slower learning performance

```python
# === Grid Search Parameters ===
nn_hidden_layers_list = [2, 4]
nn_nodes_list = [4, 6]
nn_activation_list = ['relu', 'tanh']
nn_dropout_list = [0.5, 0.0]  # 50%, 100% (no dropout)
nn_batch_size_list = [100, 10000]
nn_epochs = 20
nn_results_path = r'C:\Users\nxm240006\Downloads\nn_grid_results.csv'
```

```python
# === Build Neural Network Model ===
def build_model_nn(n_hidden, n_nodes, activation, dropout, input_dim):
    model_nn = Sequential()
    model_nn.add(Dense(n_nodes, activation=activation, input_dim=input_dim))
    if dropout > 0:
        model_nn.add(Dropout(dropout))
    for _ in range(n_hidden - 1):
        model_nn.add(Dense(n_nodes, activation=activation))
        if dropout > 0:
            model_nn.add(Dropout(dropout))
    model_nn.add(Dense(1, activation='sigmoid'))
    model_nn.compile(optimizer=Adam(), loss=BinaryCrossentropy(), metrics=[])
    return model_nn


# === Begin Grid Search ===
for nn_hl in nn_hidden_layers_list:
    for nn_node in nn_nodes_list:
        for nn_act in nn_activation_list:
            for nn_drop in nn_dropout_list:
                for nn_bs in nn_batch_size_list:
                    # ... training & evaluation code ...

                    try:
                        auc_train_nn = roc_auc_score(train_y, model_nn.predict(X_train_imputed).ravel())
                        auc_test1_nn = roc_auc_score(test1_y, model_nn.predict(X_test1_imputed).ravel())
                        auc_test2_nn = roc_auc_score(test2_y, model_nn.predict(X_test2_imputed).ravel())
                    except Exception as e:
                        print(f"Error in AUC calculation: {e}")
                        auc_train_nn, auc_test1_nn, auc_test2_nn = np.nan, np.nan, np.nan

                    # Create and append the result immediately
                    new_row_nn = pd.DataFrame([{
                        'HL': nn_hl,
                        '# Node': nn_node,
                        'Activation Function': nn_act,
                        'Dropout': f"{int(nn_drop*100)}%",
                        'Batch Size': nn_bs,
                        'AUC Train': auc_train_nn,
                        'AUC Test1': auc_test1_nn,
                        'AUC Test2': auc_test2_nn
                    }])

                    result_df_nn = pd.concat([result_df_nn, new_row_nn], ignore_index=True)
```

14.Neural Network – Grid Search

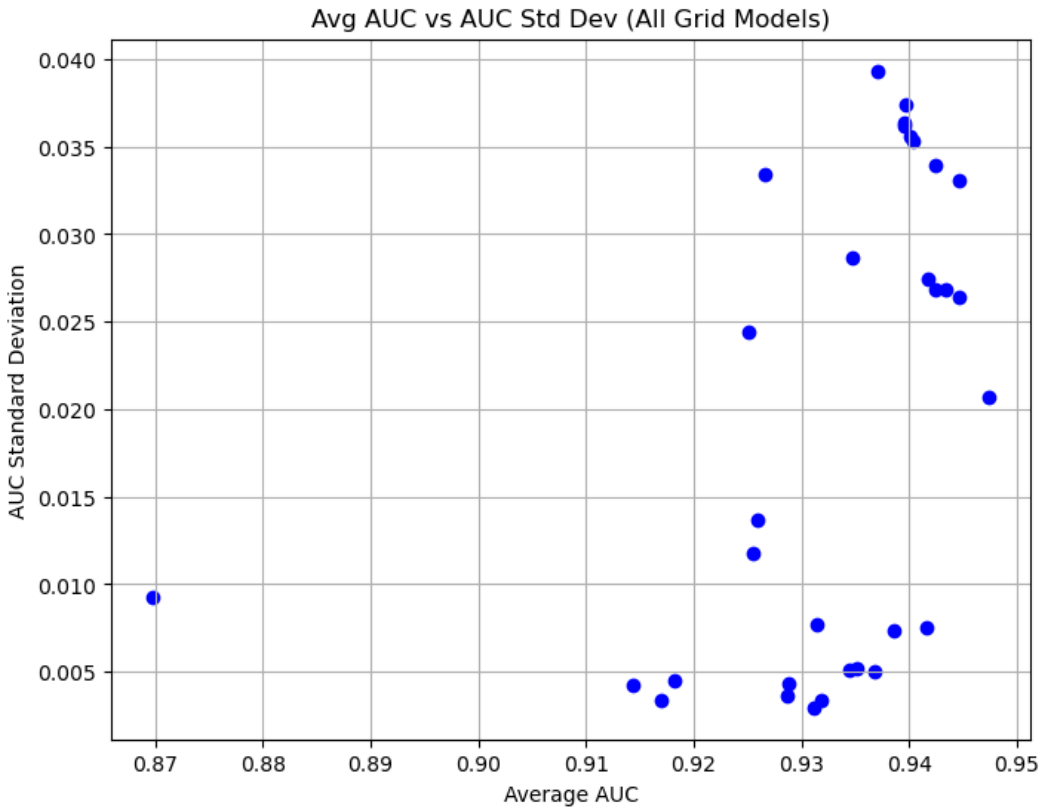
Based on **Scatterplot 1**, we would likely choose a model that is closer to the bottom right of the graph (highest avg AUC & lowest St. Dev. AUC)

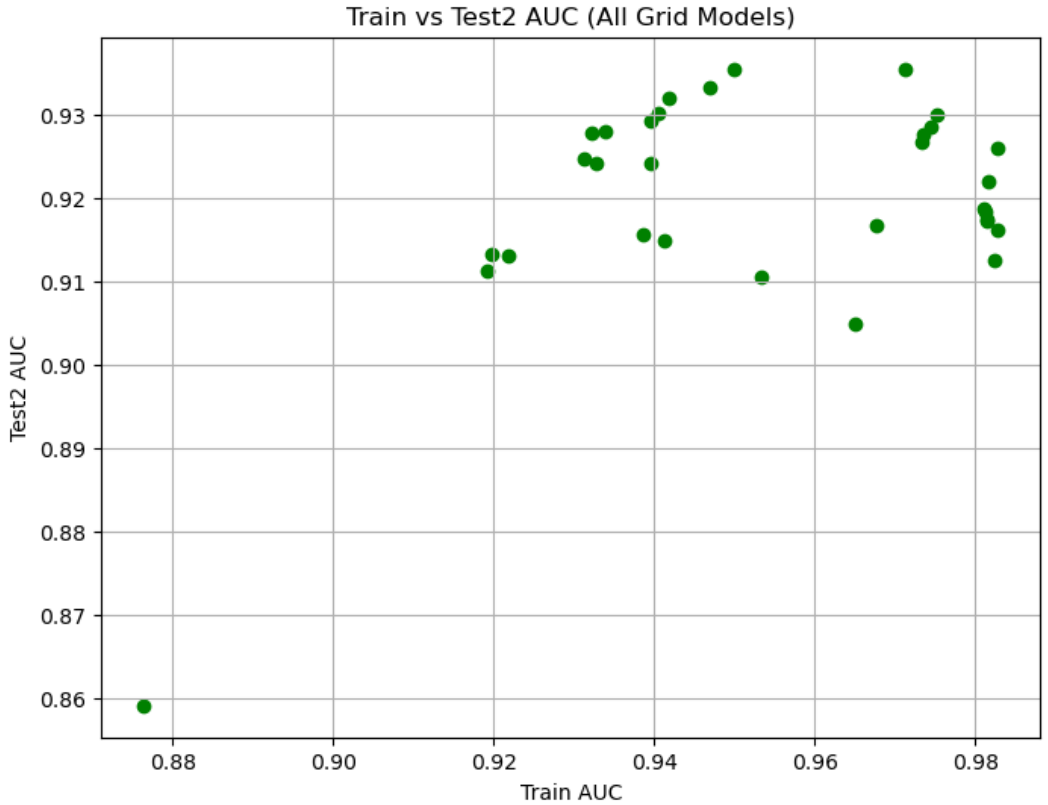
- Low St.Dev AUC Across Train + Test Samples = Low Bias/Variance
- High average AUC Across Train + Test Samples = High model classification performance

---

Based on **Scatterplot 2**, we would likely choose a mode that has the lowest difference between Train AUC and Test 2 AUC at the center.

- Prefer a low difference between Test/Train AUC to indicate low model bias





15.Neural Network – Final Model

NN Final Model Parameters	
Hidden Layers	2
# Nodes	6
Activation Function	Tanh
Dropout	0%
Batch Size	10000

AUCs of Optimal NN Model	
AUC Train	0.9419
AUC Test 1	0.9365
AUC Test 2	0.9319
AVG AUC Test	0.9342
AUC Test Variance	0.000025

16.Final Model

Given AUCs from both optimal XGB and NN models, we will choose our XGB as our final model.

Higher AUCs across Train, Test 1, and Test 2 sets

Lower AUC Test Variance = less model variance

AUCs of Optimal XGB Model	
AUC Train	0.950
AUC Test 1	0.945
AUC Test 2	0.942
AVG AUC Test	0.944
AUC Test Variance	0.000017

AUCs of Optimal NN Model	
AUC Train	0.9419
AUC Test 1	0.9365
AUC Test 2	0.9319
AVG AUC Test	0.9342
AUC Test Variance	0.000025

17.Strategy

Conservative Threshold: 0.5 (Stays below Default Rate at 10%, and involves less risk for the business)
Higher Threshold = Accepting More Applicants = Increasing the Risk
Aggressive Threshold: 0.9 (Maximizes revenue while capping right below of Default Rate at 10%)

	Train			Test 1			Test 2		
Threshold	# Total	Default Rate	Revenue	# Total	Default Rate	Revenue	#Total	Default Rate	Revenue
.1	31213	0	406	6573	0.008	84	6564	0.009	85
.2	34477	0	444	7255	0.012	92	7206	0.013	92
.3	36465	0	471	7688	0.015	95	7660	0.019	97
.4	37989	0.00005	489	8045	0.019	98	8006	0.023	99
.5	39340	0.0006	502	8335	0.024	100	8335	0.028	105
.6	41331	0.004	524	8858	0.035	105	8799	0.040	109
.7	42082	0.007	532	9052	0.040	109	8992	0.046	111
.8	44201	0.021	550	9558	0.054	112	9434	0.059	114
.9	48228	0.059	579	10415	0.085	118	10345	0.091	122